IncidentLens: An Evidence-Grounded AI Incident Investigator for OpenTelemetry Data

Project document - Working title - Prepared October 2026 - Status: Planning - Stack: Java 21, Spring Boot, PostgreSQL

1. Executive summary

IncidentLens is a backend service that performs first-pass triage of production incidents. Given an alert (for example, "checkout latency is high") and a window of OpenTelemetry (OTel) telemetry, an LLM agent calls bounded query tools over traces and metrics, forms ranked root-cause hypotheses, and produces a report in which **every claim cites stored evidence**. A validator rejects claims whose evidence does not exist or does not support the claim. An evaluation harness replays recorded fault scenarios with known ground truth and reports measured accuracy, grounding quality, cost, and latency, including a non-AI heuristic baseline for honest comparison.

The project is deliberately scoped around three things a recruiter or interviewer can verify: (1) a clear system design, (2) measurable results from an evaluation harness, and (3) honest discussion of what failed.

What it is not: it is not a replacement for commercial observability or AIOps products, it does not run against live production infrastructure in the MVP, and it makes no production-readiness claims. It is a focused, well-tested engineering project.

2. Why this project (and why it complements NotifyHub)

	NotifyHub (existing)	IncidentLens (this project)
Nature	Asynchronous messaging and delivery	Read-heavy analysis plus an AI agent
Core challenge	Delivery, retries, throughput	Tool design, grounding, evaluation, query performance
New concepts	(your current stack)	OpenTelemetry, telemetry data modeling, LLM tool calling, AI evaluation
Language	Java/Spring	Java/Spring (deliberate: spend time on new concepts, not a new language)

Verified signal (from job listings reviewed in early October 2026): backend postings commonly mention distributed tracing, metrics, and observability, and AI-backend postings mention agents, RAG, vector databases, LLM observability, and monitoring LLM performance in production. Most listings I saw were senior and outside India, so treat this as directional, not as a guarantee for campus hiring.

Assumption (not verified): many campus loops weigh DSA and CS fundamentals above projects. This project helps in project-discussion rounds and for infrastructure or AI-leaning roles. It does not replace DSA practice.

3. Problem statement

On-call engineers lose time on repetitive triage:

- **Too much data.** Thousands of traces and metric series; it is unclear where to look first.
- **Slow correlation.** Linking "latency rose at 14:02" to a failing dependency, a deploy, or saturation is manual.
- **Inconsistent quality.** Triage depends on who is on call.
- **Untrustworthy AI.** A chatbot that guesses without evidence is risky during an incident.

Target users: on-call engineers and SRE/platform teams. In a portfolio context, the "user" is an interviewer reading your repo and watching your demo.

4. Goals and non-goals

Goals

1. Produce ranked root-cause hypotheses with citations to stored evidence.
2. Guarantee by construction that every cited evidence ID exists and belongs to that investigation.
3. Measure quality with a reproducible evaluation harness over recorded incidents.
4. Keep cost and latency bounded and reported.
5. Be demonstrable in a live demo in under five minutes.

Non-goals (MVP)

- Live production ingestion at scale.
- Log analysis (stretch only).
- Auto-remediation.
- Multi-tenant SaaS features, authentication beyond a simple API key.
- Training or fine-tuning models.

5. Scope

5.1 Essential MVP

- Import recorded OTel traces and metrics ("incident bundles") into PostgreSQL.
- Derived service dependency graph.
- Six to eight bounded query tools.
- Agent loop with tool calling, step, token, and time budgets.
- Structured report schema with evidence citations and a validator.
- Evaluation harness with at least 12 recorded scenarios, a heuristic baseline, and a scripted fake LLM for deterministic CI.
- REST API, a minimal report view (simple HTML or Swagger-driven), Docker Compose setup.
- Self-instrumentation of IncidentLens with OTel.

5.2 Stretch goals (only after the MVP is stable)

- Live OTLP ingestion endpoint.
- Logs as a third signal.
- Change-event correlation (deploys, feature flags) as a tool.
- ClickHouse backend for comparison benchmarks.
- Multi-model comparison in the eval harness.
- Investigation replay and a regression dashboard over eval history.

6. Data sourcing plan (the biggest feasibility risk)

You should not build a faulty microservices system yourself. Plan:

1. **Preferred:** run an existing open-source demo application that emits OTel data, trigger several faults, and record the output with the Collector's file exporter. I believe the OpenTelemetry Demo supports fault injection through feature flags, but **I have not verified this. Confirm on Day 1** by checking its current documentation and flags.
2. **Fallback (decide by end of Day 2):** write a small synthetic telemetry generator that produces realistic traces and metrics with injected faults (slow dependency, error spike, saturation, bad deploy, traffic surge). Clearly label these scenarios as synthetic in the README.

3 Mix is acceptable: some recorded, some synthetic, each labeled.

Each scenario is stored as an **incident bundle**: telemetry files plus a `scenario.yaml` containing the alert text, time window, and **ground truth** (root-cause service and cause category).

7. Functional requirements

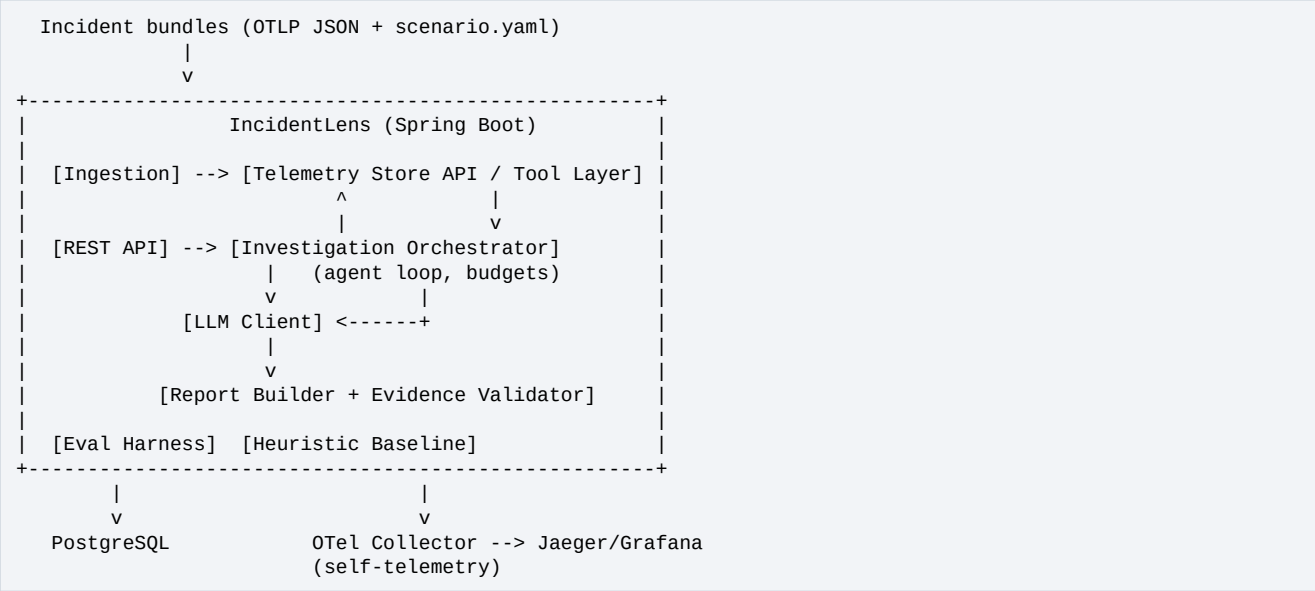
ID	Requirement
FR1	Import an incident bundle (OTLP JSON) into the database.
FR2	Derive service-to-service edges from span parent/child relationships.
FR3	Create an investigation from an alert payload, with an idempotency key to prevent duplicates.
FR4	Run the agent loop, persisting every tool call, result, and evidence item.
FR5	Enforce budgets: max tool calls, max tokens, max wall-clock time.
FR6	Produce a structured report: ranked hypotheses, cause category, confidence, summary, evidence IDs.
FR7	Validate the report; reject or flag unsupported claims.
FR8	Expose investigation status and the full step history through the API.
FR9	Run an evaluation suite and persist per-scenario results and aggregate metrics.
FR10	Run the same suite against a heuristic baseline.

8. Non-functional requirements

- **Determinism for tests:** the eval harness must be runnable with a scripted fake LLM so CI is free and repeatable.
- **Bounded cost:** every investigation reports tokens and estimated cost.
- **Query performance:** tool queries must meet a latency target you set and measure (suggested starting target: p95 under 300 ms on your largest bundle; adjust after measuring).
- **Observability:** IncidentLens emits its own traces for investigations, LLM calls, and tool calls.
- **Safety of outputs:** tool results are bounded in size and contain no secrets; raw data dumps are never sent to the model.
- **Reproducibility:** one command (`docker compose up`) starts everything.

9. System architecture

A **modular monolith**: one Spring Boot deployable with clean module boundaries. Microservices would add overhead without learning value here.



9.1 Modules

Module	Responsibility
`ingestion`	Parse OTLP JSON, normalize spans and metric points, derive service edges.
`telemetry`	Repository and query layer; the only code that touches telemetry tables.
`tools`	Tool definitions (name, JSON schema, handler), bounded output formatting, evidence ID creation.
`agent`	Orchestrator: prompt assembly, tool-calling loop, budget enforcement, retry and repair policy.
`llm`	Provider client interface, real implementation, scripted fake implementation.
`report`	Report schema, builder, and the evidence validator.
`eval`	Scenario loader, runner, scoring, baseline, result persistence.
`api`	REST controllers, request validation, error mapping.
`observability`	OTel configuration and custom spans.

9.2 Investigation flow

1. Client posts an alert (`POST /investigations`) with an idempotency key.
2. Orchestrator creates the investigation row, then starts the loop (virtual threads, so waiting on the LLM or database is cheap).
3. Loop: send system prompt, alert, and tool list to the LLM. If the LLM requests a tool, validate arguments against the tool's schema, execute with a timeout, store the result and a new **evidence item**, and return a compact summary including its `evidence_id` to the LLM.
4. Repeat until the LLM returns a final report or a budget is hit.
5. The validator checks the report. On failure it gives the LLM one repair attempt with specific feedback, then marks unsupported claims.
6. Final report and full step history are stored and served by the API.

10. Data model

PostgreSQL, with these core tables:

Table	Key columns
`bundles`	id, name, source (recorded or synthetic), window_start, window_end

Table	Key columns
`spans`	bundle_id, trace_id, span_id, parent_span_id, service, name, start_ts, duration_ns, status_code, attributes (jsonb)
`metric_points`	bundle_id, service, metric_name, ts, value, labels (jsonb)
`service_edges`	bundle_id, caller, callee, call_count, error_count, p95_ns
`investigations`	id, bundle_id, alert_text, idempotency_key (unique), status, started_at, finished_at, tokens_in, tokens_out, cost_estimate
`investigation_steps`	investigation_id, seq, type (llm_call or tool_call), request, response, latency_ms
`evidence_items`	id, investigation_id, tool_name, arguments, result_summary, structured_values (jsonb)
`reports`	investigation_id, json, validation_status
`eval_scenarios`	id, bundle_id, ground_truth_service, ground_truth_category
`eval_runs` / `eval_results`	run config, model, prompt version, per-scenario score fields

Indexing plan (measure before and after): `spans(bundle_id, service, start_ts)`, `spans(bundle_id, trace_id)`, `metric_points(bundle_id, service, metric_name, ts)`. Keep an "EXPLAIN ANALYZE before/after" note for at least two queries; it is excellent interview material.

11. Tool design

Tool outputs are the most important design decision. Principles: **bounded, aggregated, and evidence-tagged.** Each returned item carries an `evidence_id`, and each tool caps rows (for example, top 10) and rounds numbers.

Tool	Purpose
`service_overview(window)`	Per-service request count, error rate, p50/p95 latency
`top_slow_spans(service, window, limit)`	Slowest span names and their aggregate latency
`error_rate_by_service(window)`	Services ranked by error rate
`compare_windows(service, metric, baseline, incident)`	Percentage change between a baseline window and the incident window
`dependency_graph(service)`	Callers and callees with call counts and error counts
`trace_examples(service, filter, limit)`	A few representative traces, summarized by span path and durations
`metric_series(service, metric, window, step)`	Downsampled series (bounded points)
`list_change_events(window)`	Deploys or flag changes (stretch if data is available)

All tools validate arguments (time windows inside the bundle, known service names) and return structured errors the model can recover from, such as "unknown service; valid services are: ...".

12. Agent design

- **Prompting:** a system prompt defines the role, the cause categories, the evidence-citation rule, and the output JSON schema.
- **Cause categories (enum):** `slow_dependency`, `error_spike`, `resource_saturation`, `config_or_deploy_change`, `traffic_surge`, `unknown`. Allowing `unknown` is intentional; honest abstention is better than a wrong guess.
- **Budgets (starting values, tune with data):** at most 12 tool calls, a token cap per investigation, and a 90-second wall-clock limit.
- **Provider choice:** call one provider's tool-calling API directly (official SDK or plain HTTP). Frameworks such as Spring AI or LangChain4j exist, but I have not verified their current maturity, and writing the loop yourself teaches more and gives you something concrete to defend.
- **Fake LLM:** a scripted implementation that replays predefined tool-call sequences and reports. It makes CI deterministic and tests orchestration, budgets, and validation without model variance.

13. Grounding and validation

This is the project's main differentiator.

Report schema (simplified): a list of hypotheses, each with `service`, `cause_category`, `confidence` (low, medium, high), `summary`, and `evidence_ids`.

Validator checks

1. Every cited `evidence_id` exists and belongs to this investigation.
2. Numeric claims in the summary (for example, "p95 rose 340%") match the evidence's `structured_values` within a tolerance.
3. The named service appears in the cited evidence.
4. Hypotheses with no evidence are marked `unsupported` and excluded from the ranked output.

Be careful in how you describe this. The validator guarantees citations are real and consistent, not that the diagnosis is correct. Correctness is what the eval harness measures. Say this explicitly in your README and interviews.

14. Evaluation methodology

Scenario set: at least 12 (ideally 15) scenarios covering all cause categories, including ambiguous cases, a no-fault case, and a case with two simultaneous anomalies. Keep a held-out subset you do not tune against, to avoid overfitting your prompts to your tests.

Metrics

Metric	Definition
Top-1 service accuracy	Highest-ranked hypothesis names the ground-truth service
Top-3 service accuracy	Ground-truth service appears in the top three
Category accuracy	Cause category matches ground truth
Grounding rate (raw)	Share of citations valid before repair
Unsupported-claim rate	Hypotheses rejected by the validator
Tool calls per investigation	Mean and p95
Cost and latency	Mean tokens, estimated cost, wall-clock time
Run-to-run variance	Repeat each scenario at least three times with the real model

Baselines

1. **Heuristic baseline (non-AI):** rank services by combined error-rate and latency change versus a baseline window. If the agent does not beat this, report that honestly. It is a valuable finding.
2. **Prompt-only baseline (optional):** the same LLM given a compact telemetry summary but no tool calls, to show what tool calling adds.

Process discipline: record every eval run with model name, prompt version, and tool version. Never report a number you did not produce. Real-model evals are run manually (cost); the fake-LLM suite runs in CI.

15. API design

Method and path	Purpose
`POST /bundles`	Import an incident bundle
`POST /investigations`	Start an investigation (idempotency key required)
`GET /investigations/{id}`	Status, report, budgets used
`GET /investigations/{id}/steps`	Full step history (LLM and tool calls)

Method and path	Purpose
`GET /investigations/{id}/evidence/{evidenceId}`	Inspect a single piece of evidence
`POST /evals/runs`	Run the suite with a chosen model and prompt version
`GET /evals/runs/{id}`	Aggregate and per-scenario results

Use consistent error responses and OpenAPI documentation.

16. Technology stack and justification

Technology	Why
Java 21, Spring Boot 3	Your strongest stack; virtual threads for IO-bound agent and tool calls; time goes to new concepts, not a new language
PostgreSQL	Relational model fits spans, edges, and investigations; jsonb for attributes; excellent for the indexing and EXPLAIN exercises
OpenTelemetry (Collector, Java SDK)	Core new technology; used both for the data you analyze and for instrumenting IncidentLens itself
Jaeger or Grafana stack	Visualize IncidentLens's own traces; useful for the demo
LLM tool calling (one provider)	The agent capability; provider chosen after checking current API docs and pricing
JUnit 5, Testcontainers	Real-database integration tests
Docker Compose	One-command reproducible setup
Deliberately excluded	Kafka (no streaming need in the MVP), Kubernetes (stretch only), vector databases (nothing to retrieve semantically here), microservices, and agent frameworks that hide the loop

17. Failure scenarios and handling

Failure	Handling
LLM returns malformed tool call	Return a structured error to the model; allow one correction; then fail the step
Tool exceeds timeout	Return a timeout error to the model; record it; continue within budget
Provider 429 or 5xx	Retry with exponential backoff and jitter; then mark the investigation failed with a partial history
Context grows too large	Tools return summaries; truncate older steps; cap evidence payloads
Agent loops without progress	Detect repeated identical tool calls; stop and report `unknown`
Hallucinated evidence	Validator rejects; one repair attempt; unsupported claims flagged
Duplicate alert delivered twice	Idempotency key returns the existing investigation
Empty or sparse telemetry	Tools return explicit "no data"; agent should answer `unknown`
Budget exhausted	Return a partial report clearly marked incomplete

Write a test for each row. The table itself belongs in your README.

18. Testing strategy

- **Unit tests:** parsers, tool argument validation, validator logic, budget enforcement.
- **Integration tests:** Testcontainers with PostgreSQL; query correctness on a small fixed bundle.
- **Orchestrator tests:** fake LLM with scripted sequences covering malformed calls, timeouts, repair, and budget exhaustion.
- **Eval tests:** the full suite with the fake LLM in CI; real-model runs recorded manually.
- **Performance checks:** query latency before and after indexing, measured on your largest bundle.

19. Observability of IncidentLens itself

Create one trace per investigation with child spans for each LLM call and tool call, annotated with token counts, tool name, and row counts. Expose basic metrics (investigations started, completed, failed, tokens, validation failures). Showing a trace of the investigator investigating is a strong demo moment.

20. Security and safety notes

- Simple API-key authentication for the MVP; say plainly that it is minimal.
- Telemetry can contain sensitive attributes in real systems. Describe a redaction step for attributes (even if your demo data contains none) and apply an allowlist to what tools return.
- Treat model output as untrusted: validate everything against schemas before using it.
- Tool calls are read-only. There are no mutating actions, which keeps the safety story simple.

21. Deployment and costs

- **MVP deployment:** Docker Compose, runnable locally with one command. Optionally deploy to a single small VM for a live link.
- **Fallback:** a short recorded demo video in case the live demo environment fails.
- **Costs:** I have **not verified** current cloud or LLM pricing. My unverified expectation is that a small VM costs little, and LLM spend stays small if you use the fake LLM for most development and run real-model evals sparingly. Check provider pricing pages before budgeting and record actual spend in the README.
- Kubernetes manifests are a stretch item and should not block the MVP.

22. Implementation roadmap (21 days)

Days	Phase	Deliverables
1-2	Setup and data sourcing	Repo, Docker Compose, PostgreSQL, OTel Collector running. Verify whether the demo app's fault injection works; decide recorded versus synthetic by end of Day 2.
3-5	Ingestion and data model	OTLP JSON parser, schema and migrations, service-edge derivation, first 4 scenarios captured with ground truth
6-8	Tool layer	Six to eight tools with schemas, bounds, evidence IDs, integration tests, first indexing and EXPLAIN work
9-12	Agent loop	LLM client, fake LLM, orchestrator, budgets, retries, step persistence, first real investigation end to end
13-15	Report and validator	Report schema, validator, repair loop, unsupported-claim handling
16-18	Evaluation harness	12+ scenarios, metrics, heuristic baseline, repeated runs, results stored
19	Observability	Self-instrumentation, traces for investigations, simple dashboard
20	Hardening	Failure-scenario tests, README, architecture diagram
21	Demo and buffer	Demo script, recorded video, results write-up, fixes

Honest note: Days 20-21 are thin. If you fall behind, cut stretch items and reduce tools, but protect the evaluation harness and the README. Those are what carry the resume value.

23. Measurable success criteria

Targets are goals, not promises. Report whatever you actually measure.

- At least 12 scenarios with ground truth, at least 3 of them held out and never tuned against.
- Evaluation metrics reported for the agent and the heuristic baseline side by side.

- Final reports contain zero citations to nonexistent evidence (enforced by the validator), with the raw pre-repair grounding rate also reported.
- Cost and latency per investigation documented, with mean and p95.
- A measured before-and-after query optimization on at least two queries.
- All failure-scenario table rows covered by tests.
- One command brings up the full environment, and the README reproduces the demo.

24. Risks and mitigations

Risk	Mitigation
Demo app fault injection is unavailable or awkward	Synthetic generator fallback decided by Day 2
Agent accuracy is mediocre	Report honestly; analyze failures; compare to baseline. This is still valuable.
Evaluation set is too easy and scores are meaningless	Include ambiguous, no-fault, and multi-fault cases; keep a held-out set
Scope creep into ingestion or logs	Traces and metrics only in the MVP
LLM cost or rate limits	Fake LLM for development and CI; limited real runs
Time overrun	Fixed cut list: live ingestion, logs, ClickHouse, change events

25. Repository structure and demo

Repository layout (suggested)

```
incidentlens/
  README.md            (overview, results table, demo GIF, how to run)
  docs/
    ARCHITECTURE.md    (diagram, trade-offs, rejected alternatives)
    EVALUATION.md      (methodology, metrics, results, failure analysis)
    FAILURE_MODES.md   (the table above, with test links)
  bundles/             (scenarios with scenario.yaml and ground truth)
  src/main/java/...    (modules listed in section 9.1)
  src/test/java/...
  docker-compose.yml
```

Demo script (about 5 minutes)

1. Show the architecture diagram in one minute.
2. Replay an incident bundle and start an investigation.
3. Watch the tool calls and the evidence appear in the step history.
4. Open the report and click a citation to inspect the underlying evidence.
5. Show the evaluation scoreboard against the heuristic baseline.
6. Show one failure case and explain why the agent got it wrong.
7. Show the trace of the investigation in Jaeger or Grafana.

26. Resume impact

Positioning: NotifyHub shows messaging and Java/Spring. IncidentLens adds observability, LLM tool calling, AI evaluation, and measured engineering, so your profile reads as broader instead of repeated.

Draft bullets (fill in only numbers you measured):

- Built an evidence-grounded AI incident investigator in Java/Spring Boot that analyzes OpenTelemetry traces and metrics via tool-calling agents and returns ranked root-cause hypotheses with verifiable citations.

- Designed an evaluation harness over [N] recorded and synthetic fault scenarios, achieving [X]% top-1 service accuracy versus [Y]% for a non-AI heuristic baseline, with [Z] held-out scenarios.
- Implemented an evidence validator that rejects unsupported claims, reducing invalid citations from [A]% to 0% in final reports.
- Enforced step, token, and time budgets, averaging [T] tool calls and [$C] per investigation.
- Optimized telemetry queries from [P] ms to [Q] ms through indexing and schema changes, documented with EXPLAIN ANALYZE.
- Instrumented the service with OpenTelemetry to trace every investigation, LLM call, and tool call.

27. Interview preparation

Expect to discuss:

- Why tool outputs are bounded and aggregated, and what you tried first.
- How the validator works and what it does not guarantee.
- How you designed evaluation scenarios and avoided overfitting.
- Where the agent failed and why, and how it compared with the heuristic baseline.
- Why a modular monolith instead of microservices.
- Why you wrote the tool-calling loop yourself instead of using a framework.
- Postgres indexing and query-plan decisions.
- How you would scale this to live ingestion, and what would change (streaming ingestion, columnar storage, retention).
- Idempotency of investigation creation and how that relates to your NotifyHub experience.

28. Open decisions (resolve early)

1. Recorded demo data or synthetic generator (decide by end of Day 2).
2. Which LLM provider to use (check current tool-calling docs and pricing).
3. Whether to include change events in the MVP (only if the data source provides them).
4. How large to make the largest bundle for query benchmarking.

29. Final honesty check

This project will give you real, defensible experience in OpenTelemetry, telemetry data modeling, LLM tool calling, and AI evaluation. It will not make you an observability expert or produce a production-ready AIOps product. Its resume value depends on rigor: a thin evaluation set and vague claims would make it look like another AI wrapper, while measured results, a baseline, and honest failure analysis make it stand out.